



Compilers

Guillermo A. Pérez

Lecture 1: Introduction, Grading, and Compiling!

TL;DR: This course in short

What are they?

Compilers transform (high-level) programming language programs into machine-executable instructions. More generally, they translate one **formal language** into another.

Why study them?

To understand how to ask the computer to do something efficiently, and to know what the machine is actually doing to carry out the instructions we used to implement an algorithm.

References

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Familiarity with assembler (computer systems and architecture): **MIPS**; formal language theory (languages and machines, machines and computability); programming in general (intro. to programming)

What skills will we obtain?

- theory: more formal language tools!
- practice: implement a compiler

How will these skills be useful?

- The best programmers know how their compilers work and how to best code for them.
- Word processing, prog. language compilers, etc. are all compilers!

Evaluation method (1/2)

Project: a compiler

A two-part project that accounts for 60% of your grade:

- **Soon:** teams (sent to assistant)
- First, we will compile from a fragment of C to LLVM intermediate representation (40% of the 60%)
 - broken up into **deliverables**
- We will have two **recorded** presentations, one for each part, Kasper provides a test suite

Evaluation method (2/2)

Closed-book written examination

The remaining 40%:

- You can bring **one cheat sheet**
- See Blackboard for a sample exam from previous years

Important notes

- You must pass **BOTH** the project and the exam
- You can only **pass the project during the first assessment period**
- (So focus on passing the project during the *eerste zit!*)

More on the project (1/2)

Required elements in your compiler (to pass)

- commands c2ir, c2mips
- basic types: char, int, pointer
- constants, local and global variables
- reserved words and constructs
 - types (char, int, void)
 - control structures (if, else, while, for)
 - control primitives (break, continue, return)
 - const
- ...

More on the project (1/2)

Required elements in your compiler (to pass)

- ...
- single line comments
- stdio
- function/procedure definitions and calls
- basic types as function parameters and return types (by value)
- check consistency of the type of the return statement
- basic support for 1D (static) arrays
- diagnostic error messages for semantical errors
- read-only access for variables declared const
- (only for c2mips) at least 1 optimization

More on the project (2/2)

Optional elements in your compiler (check official assignment)

- variables without initializer are automatically set to 0
- conversions: explicit and implicit casting
- errors when calling undeclared functions
- nested functions including proper scoping
- default parameters (as in C++)
- multi-dimensional array support
- dynamic array support
- assignment of complete arrays (full rows)
- display warnings when overinitializing arrays
- passing arrays as value parameters
- ...

More on the project (2/2)

Optional elements in your compiler (check official assignment)

- ...
- the switch statement (with ints only)
- inline functions
- the enum type
- pointer arithmetic
- other optimizations

Outline

1. Introduction, automatic compilers, examples
2. (Flipped) Scanning, automata, regular languages, grammars
 - Chapters 1–4 from the lecture notes
3. LLVM IR: specifications and examples
4. Top-down parsers
5. Error recovery, syntax-guided AST, and symbol table creation
6. Type checking and other semantic analysis
7. MIPS: examples and generation from AST and CFG
8. Bottom-up parsers: Canonical FSM and LR(0) parsers
9. Bottom-up parsers with look-ahead: LR(k), SLR(k), LALR(k)
10. Optimizations

Compilers

What is it?

A **compiler** is a program that processes programs and translates a source program P_s written in some source language L_s into an **equivalent** target program P_t written in some target language L_t .

Bootstrapping: What came first?

The compiler is itself a program. What language is it written in?

- A third language or
- in the source language (by hand).

The insides of a compiler

From source to target

Compilers are usually separated into two parts:

- The **front end** translates the source program into an **intermediate representation**.
- The **back end** translates the intermediate representation into the target program.

The front end

Builds an **abstract representation** of the source program

- Lexical analysis a.k.a. **scanning**
- Syntactic analysis a.k.a. **parsing**

syntax errors are usually reported and a **semantic analysis** occurs: e.g. type errors are reported

Inter-optimizations & back end

Optimizer (the middle end)

An optional part of a compiler, the (common) optimizer transforms the abstract representation of the code to improve it.

The back end

- synthesis: translate the abstract representation of the program into the target language
- target-specific optimizations/transformations occur here

Scanning example

```
1  #include <stdio.h>
2  int i = 5;
3  int f(int j) {
4      int i = j;
5      return i + 1;
6  }
7  int main() {
8      printf("Hello_world!");
9      printf("%d_d", i, f(i + 1));
10     return 0;
11 }
```

- The scanner splits/groups the above

Lexical units, lexemes, tokens

Lexical unit

The scanner splits the input into sub-strings and groups them into **lexical units**, e.g.

- identifiers, keywords
- ...

Definition (Lexeme)

A **lexeme** is an element of a lexical unit.

Definition (Token)

A **token** is a pair (id, att) where id is the identifier of a lexical unit and att is an attribute (i.e. some extra information about the token).

A symbol table (1/2)

Identifiers and declarations

If the scanner matches a variable name (identifier) more than once, the tokens should have **the same attribute!**

- keep a table with variable declarations

index	lexeme
0	<i>i</i>
1	<i>bestVariable</i>

A symbol table (2/2)

- For the code below, **scoping** is needed, so the table is delegated to the parser

```
1  int i = 5;  
2  int f(int j) {  
3      int i = j;  
4      return i + 1;  
5  }
```

Obstacles for scanning

Lexical unit specification

Here, **regular expressions** will come to the rescue.

How do we split the input into tokens

Our choice of specification will allow us to use **automata** to split the program into tokens.

Parsing example (1/2)

- The parser's input are the symbols

$$((x + y) \times 3)$$

- it, a.o, matches opening/closing parentheses above

Arithmetic expressions

Consider the language of arithmetic expressions with parentheses, identifiers, natural numbers, binary operators $+$, $-$, $\times$ and the $-$ unary operator.

Parsing example (2/2)

Parsing arithmetic expressions

A parser should verify an expression is correct iff it is of the form:

- an identifier or a natural number;
- (E) , with E a valid expression;
- $-E$, with E a valid expression;
- $E_1 \text{ op } E_2$, with $\text{op} \in \{+, -, \times\}$.

- A more complete example: infix to postfix or vice versa!

Abstract syntax trees

- The parser builds an **abstract syntax tree** for $i + y \times 5$
- Similarly for the code below

```
1 while(x > 5){  
2     x = x - 1;  
3     y = 2 * y;  
4 }
```

Trees of symbol tables

The AST can be used to create a **tree of symbol tables**:

- one per procedure/function plus a global one
- the symbol-table tree **respects the AST's parent relation**

Obstacles for parsing

Syntax specification

Since we intuitively want to *grow* trees from the tokenized input, **grammars** will help us here.

How do we build a parser for a grammar?

Essentially, we will use the fact that **automata with a stack**, a.k.a. **pushdown automata**, can recognize languages given as grammars.

A word on semantic analysis

Scoping and undeclared variables

The parser will build a tree of symbol tables, so we can detect when **an undeclared variable is used**.

Type checking and type control

Again because of the symbol table, we can check whether types match during assignments and operations.

Control flow

If the language allows for **jumps** (e.g. goto) then we can also check that jumps do not change scope (so that required variables are defined). Dead-code detection can also be carried out.

Synthesis

Output the target program

From the abstract representation of the input program, the back end needs to produce the output program. But before that. . .

Common optimizations

- Control flow optimizations
- Loop optimization
- Constant propagation
- Promoting parameters to references
- . . .

Intermediate representation

Why?

- Optimizations are often target-language dependent. But, some optimizations are common to all targets.
- It allows to separate the complications of reading the input program from those arising from producing the output program.

```
1  int a, b, c;  
2  if (a > b)  
3      c = 1;  
4  else  
5      c = 2;
```

LLVM IR

```
1      %tmp = icmp sgt i32 %a, %b
2      br i1 %tmp, label %iftru, label %ifflds
3 iftru: %c = 1
4      br label %end
5 ifflds: %c = 2
6      br label %end
7 end:
```

Almost assembler

- icmp sgt i32 compares two 32-bit numbers
- br i1 is a conditional jump
- br is an unconditional jump
- register names begin with %

Unlimited amount of registers!

From IR to the target language

```
1      .text
2      .globl main
3  main:
4      li $t0, 0x2      # load 2 into a
5      li $t1, 0x3      # load 3 into b
6      slt $t3, $t1, $t0 # $t3 iff $b<a
7      blez $t3, iffls  # br if $t3<=0
8  iftru: li $t3, 0x1    # load 1 into c
9      j end            # go to end
10 iffls: li $t3, 0x2    # load 2 into c
11 end:
```

Next lecture

- **FLIPPED** lesson for regular expressions, automata, grammars, and pushdown automata (Chapters 1, 2 from the lecture notes)
- Review your notes and read (at least the definitions) from **lectures notes**